





Desenvolvimento de Equipa de Robôs Humanóides Simulados

Projeto PECI - FC Portugal

Daniel Oliveira (115359)
Lázaro Sá (115884)
Ricardo Carmo (112657)
Tiago Brito (112911)
Miguel Mota (119934)
Pedro Cunha (112960)

Sob a orientação de:
Prof. Nuno Lau - IEETA/UA
Prof. Luís Paulo Reis - LIACC/UP

Junho 2026

Resumo

Este relatório apresenta o desenvolvimento de comportamentos robustos e eficazes para a Liga de Simulação 3D do RoboCup 2026, utilizando o novo simulador baseado em MuJoCo. O projeto, realizado no âmbito do programa PECI pela equipa FC Portugal, foca na geração automática de comportamentos através de técnicas avançadas de aprendizagem por reforço profundo e otimização baseada em simulação física.

Os comportamentos desenvolvidos e documentados abrangem múltiplos níveis hierárquicos: (i) nível primitivo e de controlo motor de baixo nível, com ações como queda controlada, drible segmentado por metas, remates curtos para passe, remates longos para golo e técnicas de defesa e mergulho para guarda-redes; (ii) nível de planeamento geométrico com navegação computacional via algoritmo A*; (iii) nível tático e estratégico com decisões coordenadas. A integração de políticas contínuas e planeadores discretos no simulador MuJoCo representa um marco crucial para mitigar o impacto da transição tecnológica da liga.

Palavras-chave: RoboCup, Simulação 3D, Aprendizagem por Reforço Profundo, Robôs Humanóides, Futebol Robótico, MuJoCo, PPO, Algoritmo A*, Comportamentos Autónomos.

Contents

1 Introdução

1.1 Contexto e Motivação

O Futebol Robótico representa atualmente um dos principais ambientes de teste nas áreas de Inteligência Artificial Distribuída, Robótica Autónoma e Sistemas Multi-Agentes. Entre as ligas de Futebol do RoboCup (`https://www.robocup.org`), a Liga de Simulação 3D (*3D Soccer Simulation League*) destaca-se pelo seu foco em problemas algorítmicos fundamentais de construção de um agente inteligente e autónomo capaz de atuar em ambientes dinâmicos e competitivos.

Na Liga de Simulação 3D, cada agente robótico toma decisões de forma autónoma, mas deve contribuir para os objetivos globais da equipa (marcar golos e evitar sofrê-los). Isto requer coordenação efetiva, comunicação implícita e execução robusta de estratégias de jogo. A federação RoboCup organiza regularmente campeonatos internacionais de robótica que reúnem equipas das principais universidades e centros de investigação do mundo.

1.2 Histórico e Conquistas

Equipas de investigação do IEETA/UA (Instituto de Engenharia Eletrónica e Telemática da Universidade de Aveiro) e LIACC/UP (Laboratório de Inteligência Artificial e Ciência de Computadores da Universidade do Porto) têm participado de forma conjunta na liga de simulação 3D através da equipa FC Portugal. Esta equipa alcançou resultados de excelência ao mais alto nível mundial, incluindo quatro títulos de campeão mundial consecutivos: 2022 (Bangkok, Tailândia), 2023 (Bordéus, França), 2024 (Eindhoven, Países Baixos) e 2025 (Salvador, Brasil).

1.3 Motivação para o Novo Simulador

Uma mudança paradigmática ocorreu em 2026 quando a Liga de Simulação 3D do RoboCup migrou de forma oficial para um novo simulador baseado em MuJoCo (*Multi-Joint dynamics with Contact*). O simulador anterior, utilizado há mais de uma década, foi substituído para oferecer capacidades superiores de física realista, suporte nativo para dinâmica de contactos complexos e uma integração direta com ecossistemas modernos de aprendizagem automática.

Esta transição representa simultaneamente um desafio e uma oportunidade: um desafio porque invalida e requer a reaprendizagem completa de todos os comportamentos de locomoção e manipulação de bola anteriormente desenvolvidos; uma oportunidade porque permite explorar novas técnicas de otimização e arquiteturas de políticas que podem levar a comportamentos mais estáveis, fluidos e eficientes.

1.4 Objetivos do Projeto

Este projeto foca-se no desenvolvimento, treino e integração de novos comportamentos inteligentes e camadas de navegação para robôs humanóides simulados sob o motor de física MuJoCo. Os objetivos específicos estruturam-se em:

1. Desenvolver um ecossistema estável de ambientes baseados em Gymnasium para treino de tarefas motoras.
2. Implementar e otimizar comportamentos primitivos estáveis (queda, drible, remates curtos e longos, defesa) utilizando Aprendizagem por Reforço Profundo.
3. Desenvolver uma camada independente de planeamento de caminhos via A* para navegação em campo com desvio de obstáculos.
4. Estabelecer um mecanismo de transição e sequenciação suave entre as diferentes habilidades.
5. Validar o desempenho do agente integrado em ambientes de jogo competitivos.

2 Objetivos e Plano de Trabalho

2.1 Objetivos Gerais

Este projeto visa dotar a equipa FC Portugal com uma nova árvore de competências de baixo nível e uma camada de navegação abstrata perfeitamente adaptadas ao ecossistema MuJoCo, tendo em vista a participação competitiva no RoboCup 2026, a decorrer em Lisboa, Portugal.

O conjunto de componentes foco deste desenvolvimento engloba:

- **Queda Controlada (*Fall Down*):** Minimização do tempo de impacto e estabilização rápida do corpo no plano.
- **Mergulho do Guarda-Redes (*Diving Goalkeeper*):** Projeção lateral dinâmica para interceção de trajetórias de remates à baliza.
- **Drible por Metas (*Dribble Target*):** Condução controlada da bola ao longo de uma sequência de coordenadas dinâmicas.
- **Remate Curto e Passe (*Short Kick*):** Transferência precisa de energia para deslocamento controlado da bola a curta e média distância.
- **Remate Longo Pernas de Esparguete (*Strong Shoot*):** Extensão e aceleração máxima das juntas da perna para maximizar o deslocamento da bola em direção ao golo.
- **Camada de Planeamento A* (*A* Navigation*):** Algoritmo clássico de caminho ótimo discretizado focado em trajetórias livres de colisões com adversários.

A nível estrutural, o projeto foca-se na parametrização rigorosa dos espaços de estados e na modelação de funções de recompensa densas (*reward shaping*), garantindo a convergência acelerada das políticas sob o algoritmo PPO.

2.2 Plano de Trabalho Detalhado

O projeto iniciou-se com o enquadramento do novo simulador *rcssserver*, incluindo a análise das suas propriedades físicas, a configuração da infraestrutura distribuída de treino, a definição de uma arquitetura hierárquica e a documentação dos canais de comunicação com o agente. Atualmente, encontra-se na fase de validação e otimização fina dos hiperparâmetros de treino.

O cronograma do projeto está estruturado nos seguintes marcos cruciais:

Table 1: Cronograma do Projeto

Data	**Marco**	**Descrição**
25 Fev 2026	M1	Enquadramento inicial, especificação e planeamento
11 Mar 2026	M2	Levantamento de requisitos, estado da arte e testes iniciais
15 Abr 2026	M3	Protótipo funcional e validação individual de ambientes
23-25 Abr 2026	Nacional	Participação no Festival Nacional de Robótica para validação
20 Mai 2026	M4	Conclusão do treino das funcionalidades core e integração
3 Jun 2026	Students@DETI	Demonstração e apresentação pública no evento Students@DETI
8 Jun 2026	Relatório Técnico	Submissão da documentação e relatório técnico final
30 Jun - 6 Jul 2026	Mundial	Participação no campeonato mundial RoboCup 2026 (Incheon)

3 Estado da Arte

3.1 Aprendizagem por Reforço Profundo

A Aprendizagem por Reforço (*Reinforcement Learning - RL*) assenta na formalização matemática de Processos de Decisão de Markov (MDP), onde um agente interage com um ambiente através de ações, transitando de estado e recebendo um sinal escalar de recompensa. O objetivo fundamental consiste em computar uma política $\pi(a|s)$ que maximize a expectativa do retorno acumulado descontado no tempo.

Com o advento da Aprendizagem por Reforço Profundo (*Deep RL*), redes neuronais densas e recorrentes passaram a atuar como aproximadores universais de funções para modelar políticas e funções de valor. No âmbito deste projeto, a escolha recai sobre o algoritmo **PPO (Proximal Policy Optimization)**, um método do tipo *Actor-Critic* baseado em regiões de confiança simplificadas através de uma função objetivo de corte (*clipped surrogate objective*). O PPO destaca-se na robótica pela sua estabilidade de convergência, eficiência amostral e menor sensibilidade à oscilação abrupta de gradientes em espaços de ação contínuos.

3.2 Robótica Humanóide Simulada e o Motor MuJoCo

A simulação aplicada a estruturas humanóides exige um elevado rigor matemático na resolução das equações de dinâmica direta e inversa. O motor de física **MuJoCo** resolve as restrições de contacto e juntas através de uma formulação convexa baseada em coordenadas generalizadas, permitindo simular com precisão fenómenos como fricção seca, elasticidade de atuadores e impactos sem introduzir instabilidades numéricas severas. Isto torna o ambiente computacionalmente eficiente para algoritmos que requerem milhões de iterações de amostragem.

3.3 A Liga de Simulação 3D no RoboCup

Historicamente caracterizada pelo uso do servidor *SimSpark*, a liga introduziu em 2026 a plataforma *rcssservermj*, unificando as regras competitivas do futebol multi-agente com a precisão física do MuJoCo. Cada equipa controla múltiplos robôs humanóides que operam de forma assíncrona, exigindo que os comportamentos primitivos executados localmente por cada agente sejam robustos a perturbações e capazes de responder em tempo real a comandos táticos abstratos de alto nível.

3.4 Pesquisa de Caminhos com o Algoritmo A*

O algoritmo A* é um método de pesquisa informada em grafos que determina o caminho de custo mínimo entre um nó de origem e um destino. Em cada iteração, expande o nó n com menor valor da função de avaliação:

$$f(n) = g(n) + h(n) \tag{1}$$

onde $g(n)$ é o custo acumulado do caminho percorrido desde a origem até n, e $h(n)$ é uma heurística que estima o custo remanescente até ao destino. O algoritmo mantém uma *open list* com os nós a explorar (ordenada por f) e uma *closed list* com os nós já expandidos, terminando quando o destino é retirado da *open list*.

A qualidade da heurística determina a eficiência e a otimalidade do resultado. Uma heurística diz-se *admissível* se nunca sobrestimar o custo real; nesse caso, o A* garante encontrar o caminho ótimo. Em espaços de navegação bidimensionais, a distância euclidiana ao destino é uma escolha comum e admissível. Uma heurística mais próxima do custo real reduz o número de nós expandidos, aproximando o comportamento do A* de uma pesquisa dirigida ao objetivo.

4 Arquitetura do Sistema e Comportamentos Primitivos

4.1 Visão Geral da Infraestrutura

O ecossistema de desenvolvimento e controlo do agente segue uma arquitetura hierárquica dividida em camadas principais: a camada de treino (*Training Layer*), responsável pela otimização via algoritmo PPO da biblioteca *Stable-Baselines3*; a camada de mundo (*World Layer*), que computa a semântica de jogo e as posições relativas; e a camada de simulação (*Simulation Layer*), onde corre o servidor central *rcssservermj* baseado em MuJoCo, encarregue de processar os contactos e restrições mecânicas a uma frequência de amostragem controlada.

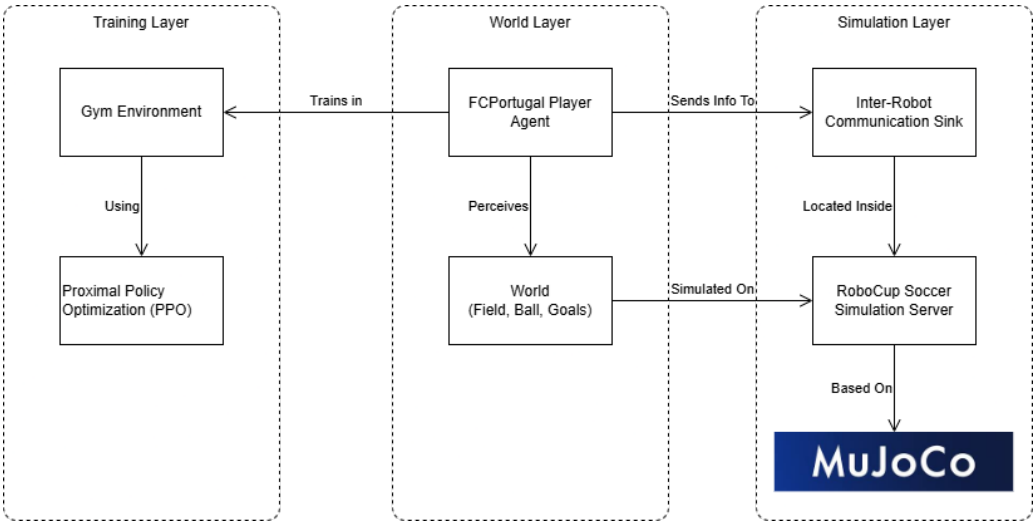


Figure 1: Arquitetura hierárquica e fluxo de dados do sistema

4.2 Especificação Detalhada das Habilidades Motores (Treino por RL)

Nas subsecções seguintes, detalha-se a modelação matemática e computacional de cada um dos comportamentos primitivos motores baseados em aprendizagem por reforço profundo.

4.2.1 Comportamento de Queda Controlada (*Fall Down*)

- **Environment:** O robô é inicializado numa postura vertical instável com uma perturbação e inclinação angular aleatória aplicada ao tronco no eixo Y (entre $-30°$ e $+30°$), forçando-o a reagir a desequilíbrios realistas. O horizonte do episódio está limitado a 150 passos a 50 Hz.

- **Observation Space:** Vetor contínuo composto pela orientação do tronco expressa em ângulos de Euler (ou quaternions), as velocidades angulares do giroscópio do robô, as posições angulares atuais dos servos das articulações e o vetor de aceleração linear do acelerómetro central.

- **Reward Function:** Baseia-se numa penalização puramente temporal. Aplica-se uma recompensa estritamente negativa constante de -0.02 por cada passo de

simulação em que o robô permanece acima do limiar horizontal de queda. O episódio termina imediatamente assim que o tronco, cabeça e extremidades atingem uma altura inferior ao limiar crítico plano, incentivando a política a minimizar o tempo total de impacto.

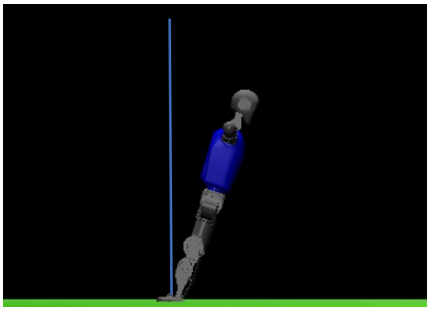


Figure 2: Configuração inicial de inclinação no ambiente de queda

4.2.2 Mergulho do Guarda-Redes (*Goalkeeper.py*)

- **Environment:** O ambiente de guarda-redes foi desenvolvido para treinar uma política de defesa lateral contra remates dirigidos à baliza da equipa. O robô é inicializado na posição $(-26.5,\ 0.0,\ 0.625)$ m, orientado para o campo, enquanto o plano da baliza se encontra em $X = -30.0$ m. Cada episódio tem um limite máximo de 300 passos de simulação a 50 Hz.

  A bola é inicializada em repouso numa posição amostrada entre $X \in [-20.0,\ -15.0]$ m e $Y \in [-8.0,\ 8.0]$ m, sendo posteriormente disparada após um atraso aleatório entre 1.0 e 3.0 s. Esta pausa obriga o agente a antecipar a trajetória antes do remate efetivo, em vez de reagir apenas no instante de contacto. O alvo do remate é amostrado maioritariamente dentro da zona da baliza ($Y \in [-1.35,\ 1.35]$ m), mas inclui também remates mais abertos ($Y \in [-3.0,\ 3.0]$ m) para aumentar a diversidade dos cenários. A velocidade horizontal da bola varia entre 6.0 e 12.0 m/s.

  O ambiente mantém internamente um modelo MuJoCo separado para previsão da trajetória da bola. A partir da posição e velocidade atuais, este modelo simula a evolução da bola com passo temporal de 0.005 s até ao plano da baliza ou até ao horizonte máximo de previsão, permitindo estimar o ponto de chegada, a linha defensiva mais provável e um ponto de bloqueio à frente do guarda-redes. Antes do mergulho, o robô usa a skill de locomoção *Walk* para se deslocar lateralmente até à linha de interceção prevista. Quando o remate já foi libertado, a bola segue em direção à baliza e a distância bola-guarda-redes é inferior a 10.0 m, é ativada uma fase de defesa em que a política PPO é combinada com uma ação assistida de mergulho.

  A ação assistida divide o mergulho em três fases: impulso inicial, extensão em voo e absorção da queda. Nesta fase, os 23 atuadores do robô continuam a ser controlados pela política, mas a ação aprendida é misturada com uma ação heurística de mergulho e, em seguida, o tronco, braços e pernas são reajustados para uma postura de bloqueio com as mãos projetadas para a frente e para cima. O episódio termina quando a bola cruza o plano da baliza dentro da janela de golo, quando cruza a baliza fora dessa janela, ou quando é atingido o limite temporal.

- **Observation Space:** O espaço de observação é composto por 77 dimensões contínuas, normalizadas para o intervalo $[-1,\ 1]$, e organizado nos seguintes grupos:
  - *Estado articular* (46 dimensões): posições normalizadas dos 23 motores relativamente aos limites articulares de cada junta, e velocidades articulares normalizadas por 25 rad/s.
  - *Estado inercial* (10 dimensões): velocidade angular do tronco medida pelo giroscópio, normalizada por 10 rad/s; vetor de gravidade projetado no referencial do corpo; e velocidade linear do centro de massa no referencial do corpo, normalizada por 3 m/s.
  - *Altura* (1 dimensão): coordenada z da posição global do robô, usada como indicador compacto de estabilidade e queda.
  - *Estado da bola* (6 dimensões): posição relativa da bola no referencial do corpo, normalizada por 8 m, e velocidade da bola estimada por diferenciação temporal, também projetada no referencial do corpo e normalizada por 8 m/s.
  - *Indicação do lado de defesa* (1 dimensão): sinal $\{-1,\ +1\}$ que indica se o remate previsto deve ser defendido para o lado negativo ou positivo do eixo lateral Y.
  - *Ação anterior* (23 dimensões): ação comandada no passo anterior durante a fase ativa de defesa, permitindo à política modelar a continuidade temporal do movimento de mergulho.

- **Reward Function:** A recompensa efetivamente devolvida ao PPO é centrada na qualidade do bloqueio com as mãos relativamente ao ponto de interceção previsto. Para cada passo em que a defesa está ativa, o ambiente calcula um ponto de bloqueio p_{block} a partir da previsão MuJoCo da bola e mede a distância no eixo X entre esse ponto e a mão mais próxima do robô. A pontuação de bloqueio é dada por:

$$s_{hand} = \text{clip}\left(1 - \frac{d_{hand,x}}{1.25},\ 0,\ 1\right) \tag{2}$$

A recompensa por passo é formulada como uma penalização limitada pela ausência de alinhamento entre as mãos e o ponto previsto:

$$R_{goalkeeper} = -0.20 \cdot (1 - s_{hand}) \tag{3}$$

Assim, o melhor caso aproxima-se de 0.0 quando as mãos se encontram alinhadas com o ponto de bloqueio, enquanto posições afastadas recebem recompensa negativa. Esta formulação evita bónus excessivos e transforma a tarefa numa minimização contínua do erro de interceção.

Para análise e diagnóstico, o ambiente calcula ainda várias penalizações auxiliares, registadas no *logger* mas não somadas à recompensa devolvida ao PPO: desalinhamento lateral quando o guarda-redes se desloca para o lado errado, erro face à linha defensiva prevista, penalização auxiliar derivada do alinhamento das mãos com o ponto de bloqueio, falha de contacto quando a bola cruza o plano da baliza fora da janela de golo, penalização por golo sofrido e indicador de queda. Estas métricas permitem avaliar separadamente a qualidade do lado escolhido, a capacidade de

seguir a trajetória prevista e o resultado final da defesa, mesmo quando a recompensa de treino principal permanece focada no bloqueio com as mãos.

Quando o episódio termina, o resultado é classificado através das seguintes condições:

- **Golo sofrido**: a bola cruza $X = -30.0$ m com $|Y| \leq 1.05$ m e altura inferior a 0.8 m;
- **Remate defendido ou fora**: a bola cruza o plano da baliza fora da janela de golo, sendo registada a ocorrência ou não de contacto com o guarda-redes;
- **Fim por tempo**: o episódio atinge o limite de 300 passos sem decisão definitiva no plano da baliza.

4.2.3 Drible Orientado a Metas (*dribble_ target.py*)

- **Environment:** O ambiente de drible orientado a metas é estruturado em duas fases sequenciais que decompõem o problema de condução da bola numa abordagem progressiva.

  Na primeira fase, o robô é inicializado em $(-3.5,\ 0.0,\ 0.0)$ m e utiliza a política de locomoção recorrente *WalkGRU* para se aproximar da bola, que se encontra posicionada na origem do campo em $(0.0,\ 0.0,\ 0.0)$ m. O robô calcula dinamicamente um ponto de ancoragem atrás da bola, mantendo um desvio de aproximação inicial de 0.18 m que decresce progressivamente até um mínimo de 0.10 m à medida que o drible avança. A transição para a fase de drible ocorre quando a distância ao ponto de condução calculado é inferior a 1.00 m.

  Na segunda fase, o controlo transita para a política *walkOLD*, sendo ativado um período de graça de 25 passos para estabilização. Durante este período, o robô permanece atrás da bola sem forçar movimento frontal, evitando a perda de controlo durante a transição entre os dois controladores. Após o período de graça, a política aprende a guiar a bola ao longo de um percurso gerado proceduralmente entre a posição inicial da bola e o centro da baliza adversária. O percurso é composto por um segmento reto inicial de 0.45 m seguido de uma curva sinusoidal com amplitude de 1.0 m, cujo número de ondas escala com a distância a percorrer. Os vértices do percurso são revelados progressivamente, desvendando o próximo ponto de passagem apenas quando a bola se encontra a menos de 0.12 m do atual. O episódio termina quando a bola é perdida, o robô cai, ou quando o percurso é concluído.

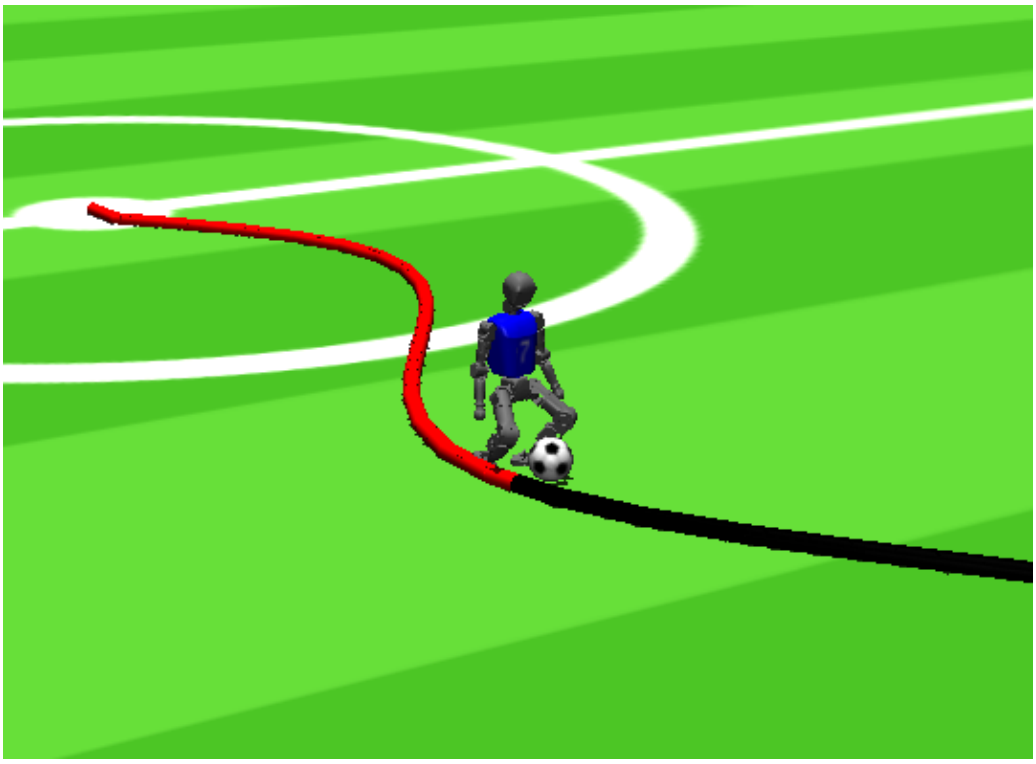


Figure 3: Visualização do ambiente *dribble_target*

- **Observation Space:** O espaço de observação é composto por 73 dimensões contínuas, normalizadas para o intervalo $[-1,\ 1]$, e organizado nos seguintes grupos:
  - *Estado articular* (44 dimensões): posições normalizadas dos 22 motores do robô, calculadas relativamente ao ponto médio do intervalo articular de cada junta, e as respetivas velocidades normalizadas por 25 rad/s.
  - *Estado inercial* (7 dimensões): velocidade angular do tronco medida pelo giroscópio, normalizada por 10 rad/s; velocidade linear do centro de massa no referencial do corpo, normalizada por 3 m/s; e altura do tronco normalizada para $[0,\ 1]$.
  - *Estado da bola* (3 dimensões): posição relativa da bola no referencial do corpo, normalizada por 4 m.
  - *Ação anterior* (3 dimensões): ação executada no passo anterior, composta pelos comandos de avanço, lateral e orientação, que fornece à política informação sobre a trajetória recente e facilita movimentos suaves.
  - *Navegação* (6 dimensões): vetor ao próximo vértice ativo do percurso no referencial do corpo, normalizado por 5 m; vetor ao vértice final (baliza) no referencial do corpo, normalizado por 8 m; e tangente unitária ao percurso num ponto de antevisão de 0.15 m à frente da bola, no referencial do corpo.
  - *Estado interno do controlador* (2 dimensões): orientação relativa interna, normalizada por 80, e taxa de rotação interna, normalizada por 18/passo.
  - *Posição dos pés* (5 dimensões): posições relativas do pé esquerdo e do pé direito no referencial do corpo (2×2 valores), e sinal de avanço do pé ativo da marcha (1 valor).

  - *Desvio lateral da bola* (1 dimensão): ângulo entre o vetor da bola ao ponto mais próximo do percurso e a tangente ao percurso, normalizado por 90, que indica o lado e a magnitude do desvio lateral da bola relativamente ao eixo central da rota.

- **Reward Function:** A função de recompensa combina sete componentes que equilibram progressão ao longo do percurso, qualidade do controlo da bola e eficiência do drible:

$$R_{dribble} = r_{prog} + r_{ctrl} + r_{lat} + r_{path} + r_{speed} + r_{dist} + r_{step} \tag{4}$$

Progressão ao longo do percurso, modulada pela qualidade do controlo:

$$r_{prog} = 10 \cdot \frac{\Delta_{progress}}{L_{path}} \cdot (0.25 + 0.75 \cdot g_{ctrl}) \tag{5}$$

onde $\Delta_{progress}$ é o avanço da bola ao longo do percurso neste passo, L_{path} é o comprimento total do percurso, e g_{ctrl} é um núcleo gaussiano que recompensa manter a bola próxima dos pés:

$$g_{ctrl} = \exp\left(-\frac{(d_{rb} - 0.16)^2}{2 \cdot 0.28^2}\right) \tag{6}$$

com d_{rb} a distância robô-bola. O núcleo atinge o máximo em $d_{rb} = 0.16$ m, penalizando implicitamente tanto a bola demasiado longe como demasiado perto.

Controlo da bola, proporcional ao núcleo gaussiano:

$$r_{ctrl} = 1.4 \cdot g_{ctrl} \tag{7}$$

Alinhamento lateral, penalizando o desvio da bola relativamente ao eixo central do percurso:

$$r_{lat} = -0.35 \cdot \tanh\left(\frac{|e_{lat}|}{0.10}\right) \tag{8}$$

onde e_{lat} é o erro lateral em metros. A função tangente hiperbólica fornece uma penalização suave e limitada, evitando gradientes excessivamente elevados para desvios grandes.

Precisão do percurso, penalizando o desvio ortogonal ao percurso de forma quadrática:

$$r_{path} = -0.45 \cdot \left(\frac{e_{path}}{0.40}\right)^2 \tag{9}$$

com limite máximo de erro tolerado de $e_{path} = 0.40$ m.

Velocidade excessiva da bola, desincentivando remates que afastem a bola do controlo do robô:

$$r_{speed} = -0.20 \cdot \max(0,\ v_{ball} - 1.2) \tag{10}$$

Aproximação progressiva, recompensando a redução da distância robô-bola:

$$r_{dist} = 0.8 \cdot \left(d_{rb}^{t-1} - d_{rb}^{t}\right) \tag{11}$$

Custo por passo, incentivando episódios mais curtos e eficientes:

$$r_{step} = -0.0005 \tag{12}$$

A recompensa total é limitada ao intervalo $[-12,\ 25]$ para estabilidade do gradiente do PPO.

No término do episódio, são aplicadas recompensas e penalizações terminais:

- **Percurso terminado**: bónus de $+20.0 + 3 \cdot \eta_{ball} + 1 \cdot \eta_{robot} + 1 \cdot \eta_{time}$, onde η_{ball} e η_{robot} medem a eficiência de percurso da bola e do robô como a razão entre a distância direta e a distância total percorrida, e η_{time} é o inverso do número de passos, premiando dribles mais rápidos;
- **Queda do robô** ($h_{tronco} < 0.3$ m): penalização de -8.0;
- **Perda de controlo da bola** ($d_{rb} > 1.20$ m ou $e_{path} > 0.80$ m, apenas após o período de graça): penalização de -6.0.

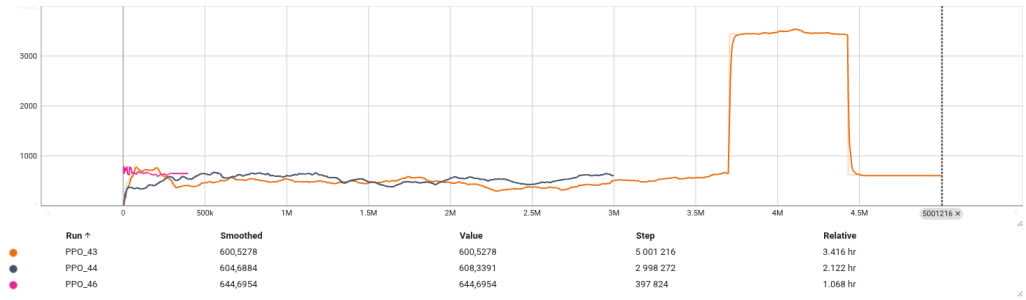


Figure 4: Evolução da recompensa média por episódio ao longo do treino PPO do agente de drible orientado a metas. A curva suavizada evidencia a progressão do agente desde a fase de aproximação até à aprendizagem do controlo eficaz da bola ao longo do percurso.

4.2.4 Remate Curto/Passe Controlado (*short_kick.py*)

- **Environment:** O ambiente de passe curto é estruturado em duas fases sequenciais que decompõem o problema do remate numa abordagem progressiva.

  Na primeira fase, o robô é inicializado na posição $(-3.5,\ 0.0,\ 0.625)$ e utiliza o comportamento *Walk* para se aproximar da bola, que se encontra posicionada a 2.5 m à sua frente. O robô calcula dinamicamente um ponto atrás da bola alinhado com o alvo, mantendo um desvio de 0.15 m. A transição para a fase de remate ocorre quando a distância ao ponto calculado anteriormente é inferior a 0.08 m e o erro de orientação é inferior a 5°.

  Na segunda fase, o controlo transita para a política *PPO*, que aprende a executar o remate dentro de um limite temporal de 150 passos. O ambiente gera um alvo aleatório a uma distância entre 3 e 7 m e com um ângulo de abertura de até 45° relativamente à posição inicial da bola, simulando passes táticos rasteiros em espaço aberto. O episódio termina quando a bola assenta após o remate, o robô cai, ou o prazo de remate é excedido.



Figure 5: Visualização do ambiente *short_kick*

- **Observation Space:** O espaço de observação é composto por 89 dimensões contínuas, normalizadas para o intervalo $[-1,\ 1]$, e organizado nos seguintes grupos:
  - *Estado articular* (44 dimensões): posições normalizadas dos 22 motores do robô, calculadas relativamente ao ponto médio do intervalo articular de cada junta, e as respetivas velocidades.
  - *Estado inercial* (7 dimensões): velocidade angular do tronco medida pelo giroscópio; velocidade linear do centro de massa no referencial do corpo; e altura do tronco.
  - *Estado da bola* (3 dimensões): posição relativa da bola no referencial do corpo.
  - *Estado do alvo* (2 dimensões): posição relativa do alvo no referencial do corpo.
  - *Ação anterior* (23 dimensões): ação executada no passo anterior, que fornece à política informação sobre a trajetória recente e facilita movimentos suaves.
  - *Velocidade da bola calculada* (3 dimensões): velocidade instantânea da bola no referencial do corpo, estimada por diferenciação temporal de posição ($\Delta P/\Delta t$) sobre uma janela de 2 posições consecutivas.
  - *Métricas de navegação* (4 dimensões): distância da bola ao alvo; distância do robô à bola; altura da bola; ângulo do robô ao alvo.
- **Reward Function:** A função de recompensa combina quatro componentes sequenciais que equilibram a qualidade do remate, a precisão da trajetória e a estabilidade do robô:

$$R_{short_kick} = r_{dir} + r_{flight} + r_{final} + r_{pen} \tag{13}$$

Bónus de direção inicial, avaliado numa janela de 2 a 8 passos após a deteção do remate (deslocamento da bola > 0.03 m), que recompensa o alinhamento entre o vetor velocidade inicial da bola e a direção ao alvo:

$$r_{dir} = \frac{B_{dir}}{5} \cdot \max\Big(0,\ \hat{v}_{bola} \cdot \hat{d}_{alvo}\Big) \tag{14}$$

com $B_{dir} = 5.0$, aplicado de forma faseada ao longo da janela temporal.

Modelação de trajetória, aplicada a cada passo após o remate e até a bola parar, que recompensa a redução incremental da distância ao alvo:

$$r_{flight} = \lambda_{flight} \cdot (d_{t-1} - d_t), \quad \lambda_{flight} = 2.0 \tag{15}$$

Recompensa terminal de precisão, calculada quando a bola para (deslocamento $<$ 0.0015 m durante 30 passos consecutivos), com base no progresso relativo da bola em direção ao alvo:

$$r_{final} = \max\left(0,\ 50 \cdot \frac{d_{inicial} - d_{final}}{d_{inicial}}\right) + B_{standing} + \mathbb{1}_{[d_{final} < r_{accept}]} \cdot B_{target} \tag{16}$$

com $B_{standing} = 60.0$ (bónus por não ter caído), $B_{target} = 60.0$ (bónus adicional se $d_{final} < 1.5$ m), e uma penalização proporcional caso a bola não tenha percorrido pelo menos 30% da distância total ao alvo.

Penalizações discretas, aplicadas por eventos de falha:

$$r_{pen} = -\delta_{fall} \cdot \mathbb{1}_{queda} - \delta_{lazy} \cdot \mathbb{1}_{sem\ remate\ antes\ do\ passo\ 150} \tag{17}$$

com $\delta_{fall} = 50.0$ e $\delta_{lazy} = 75.0$.

Quando o episódio termina, o resultado é classificado em três casos:

- **Sucesso** ($d_{final} < 1.5$ m): a bola atingiu a zona do alvo, acumulando os bónus $B_{standing}$ e B_{target};
- **Queda do robô** ($h_{tronco} < 0.3$ m): penalização de -50.0, com o episódio a terminar imediatamente depois da queda;
- **Prazo de remate excedido** (sem remate até ao passo 150): penalização adicional de -75.0, prevenindo comportamentos passivos do agente.

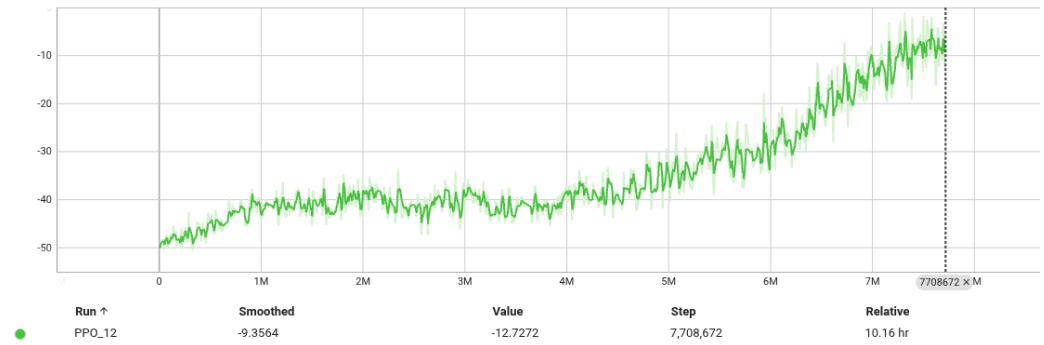


Figure 6: Evolução da recompensa média por episódio ao longo do treino PPO do agente de remate curto. A curva suavizada evidencia a aprendizagem por parte do agente o que significa uma melhoria contínua no desempenho.

4.2.5 Remate Longo (*strong_ shoot.py*)



Figure 7: Ambiente de treino do remate longo

- **Environment:** O ambiente de remate longo é composto por três fases sequenciais de execução, projetadas para decompor o problema de remate em subproblemas progressivamente mais complexos.

  Na primeira fase, o robô caminha autonomamente até uma posição de aproximação à bola utilizando uma política de locomoção recorrente pré-treinada. O robô é inicializado a uma distância amostrada aleatoriamente no intervalo $[1.8, 2.3]$ m da bola, que se encontra fixamente posicionada na origem do campo em $(0, 0, 0.042)$ m, onde 0.042 m corresponde ao raio da bola. A variabilidade da distância de spawn foi introduzida para evitar sobreajuste a uma distância fixa de aproximação, promovendo maior robustez da política de remate. A posição de aproximação é calculada de forma a posicionar o robô ligeiramente atrás e à esquerda da bola relativamente à direção de remate, maximizando o alinhamento do pé direito com o centro da bola.

  Na segunda fase, o robô abranda progressivamente através de um fator de desaceleração exponencial e assume uma pose de preparação (*wind-up pose*), com os braços elevados a 90 e uma ligeira flexão nos joelhos de 20. Esta pose foi definida manualmente com o objetivo de colocar o robô numa configuração articular estável e propícia à execução do remate, minimizando a perturbação introduzida pela transição entre a fase de locomoção e a fase de controlo por aprendizagem.

  Na terceira fase, a política de *reinforcement learning* toma controlo total dos 23 atuadores do robô e aprende a executar o movimento de remate a partir da pose de preparação. O episódio termina quando a bola para completamente após o impacto, quando o robô cai antes de contactar a bola, ou quando é atingido o limite máximo de 600 passos de simulação.

- **Observation Space:** O espaço de observação é composto por 83 dimensões contínuas, normalizadas para o intervalo $[-1, 1]$, e pode ser decomposto nos seguintes grupos:

  - *Estado articular* (46 dimensões): posições normalizadas dos 23 motores do robô, calculadas relativamente ao ponto médio do intervalo articular de cada junta, e as respetivas velocidades normalizadas por um fator de 25 rad/s. Esta normalização garante que os valores se encontram aproximadamente no intervalo $[-1, 1]$ independentemente da configuração do robô.
  - *Estado inercial* (7 dimensões): a velocidade angular do tronco medida pelo giroscópio, normalizada por 10 rad/s; o vetor de gravidade projetado no referencial do corpo, obtido através da inversão do quaternião de orientação do tronco e aplicado ao vetor gravitacional mundial $[0, 0, -1]$; e a velocidade linear do centro de massa no referencial do corpo, normalizada por 3 m/s. A projeção do vetor de gravidade é preferida à representação direta do quaternião por ser mais interpretável pela rede neuronal e invariante a rotações em torno do eixo vertical.
  - *Altura* (1 dimensão): a coordenada z do centro de massa do robô, que serve como indicador implícito do estado de equilíbrio.
  - *Estado da bola* (7 dimensões): a posição relativa da bola no referencial do corpo, normalizada por 5 m; a velocidade relativa da bola no referencial do corpo, também normalizada por 5 m/s; e a distância escalar ao centro da bola, normalizada por 5 m. A representação no referencial do corpo é utilizada para tornar a observação invariante à orientação absoluta do robô no campo. Devido a limitações do sistema de visão do robô, a posição da bola é atualizada apenas quando esta se encontra no campo de visão do robô, sendo mantido o último valor conhecido nos restantes passos.
  - *Ação anterior* (23 dimensões): a ação executada no passo anterior, que fornece à política informação sobre a trajetória articular recente e facilita a aprendizagem de movimentos suaves e coordenados, reduzindo oscilações entre passos consecutivos.

- **Reward Function:** A função de recompensa é composta por várias componentes que atuam em diferentes fases do episódio, com o objetivo de guiar a política desde a aproximação ao contacto até à maximização da distância e precisão do remate.

  Antes do contacto, o robô recebe duas recompensas de modelação (*shaping rewards*). A primeira é uma recompensa de aproximação do pé à bola, definida como:

$$r_{\text{approach}} = \exp(-6 \cdot d_{\text{foot}}) \cdot 0.3 \tag{18}$$

  onde d_{foot} é a distância euclidiana entre o pé direito e a bola. Esta recompensa incentiva a política a posicionar o pé corretamente antes do remate. A segunda é uma recompensa de estabilidade proporcional ao alinhamento vertical do tronco:

$$r_{\text{upright}} = -g_z \cdot 0.05 \tag{19}$$

  onde g_z é a componente vertical do vetor de gravidade projetado, penalizando inclinações excessivas do tronco antes do contacto.

  No momento do primeiro contacto, é atribuído um bónus proporcional à velocidade de impacto registada:

$$r_{\text{contact}} = \text{clip}(v_{\text{contact}}, 0, 10) \cdot 0.5 \tag{20}$$

Este bónus incentiva remates com maior força de impacto inicial. O quaternião de orientação do tronco no momento do contacto é capturado e fixado (*kick_ quat_ inv*), sendo utilizado em todos os cálculos subsequentes para garantir que a direção de remate é avaliada de forma consistente independentemente de rotações posteriores do tronco.

Após o contacto, a cada passo de simulação, a política recebe uma recompensa de modelação baseada na velocidade da bola no referencial do corpo no momento do impacto:

$$r_{\text{vel}} = v_{\text{fwd}} \cdot 0.09 - |v_{\text{lat}}| \cdot 0.03 \tag{21}$$

onde v_{fwd} é a componente frontal da velocidade da bola e $|v_{\text{lat}}|$ é o desvio lateral. Esta componente fornece um sinal de gradiente denso durante toda a fase de voo da bola, acelerando a convergência da política.

No término do episódio, é aplicada uma recompensa terminal gaussiana que combina distância percorrida e precisão direcional de forma aditiva:

$$R_{\text{terminal}} = \underbrace{\exp\left(-\frac{(22.5 - d_{\text{fwd}})^2}{2 \cdot 6^2}\right) \cdot 60}_{\text{distância}} + \underbrace{\exp\left(-\frac{d_{\text{lat}}^2}{2 \cdot 0.75^2}\right) \cdot 40}_{\text{direção}} \tag{22}$$

onde d_{fwd} é o deslocamento frontal final da bola no referencial fixado no momento do impacto e d_{lat} é o desvio lateral. O objetivo virtual de 22.5 m define o alvo de distância da recompensa gaussiana, criando um gradiente suave mesmo quando a bola fica muito aquém do objetivo, evitando recompensas esparsas que dificultariam a convergência da política. A componente direcional penaliza remates com desvio lateral excessivo independentemente da distância alcançada.

Adicionalmente, é aplicada uma penalização de -10 por passo sempre que a bola se desloca para trás relativamente à direção de remate, desincentivando remates invertidos, e uma penalização de -20 em caso de queda do robô antes do contacto com a bola, garantindo que a política mantém o equilíbrio durante a fase de preparação.

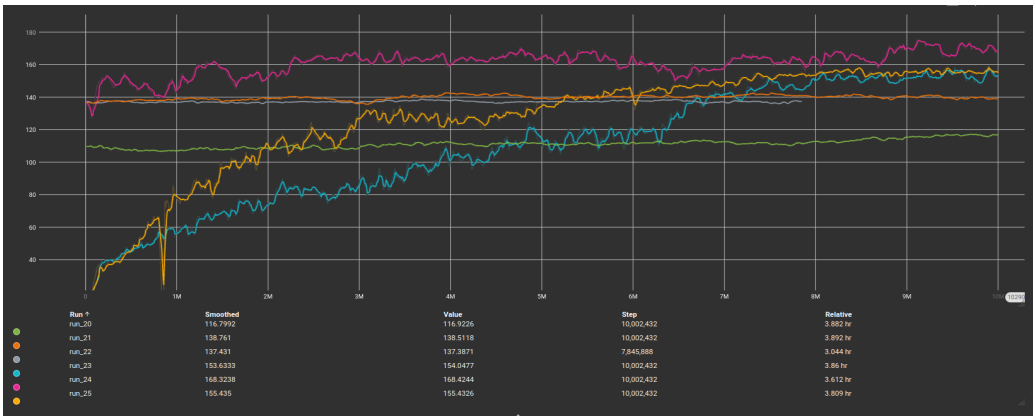


Figure 8: Evolução da recompensa média por episódio ao longo do treino PPO do agente de remate longo.

4.3 Mecanismo de Transição de Comportamentos

De modo a garantir uma alternância estável e contínua entre os diversos comportamentos, estabeleceu-se o comportamento *Walk* como o estado inicial padrão para todos eles.

5 Camada de Navegação

A navegação autónoma do agente (*astar_ navigation.py*) é implementada como uma camada algorítmica independente, invocada a cada ciclo de execução. O seu objetivo é decompor um destino global em *waypoints* intermédios livres de colisões, devolvendo ao agente o próximo ponto a atingir e a orientação desejada.

5.1 Discretização do Espaço

A cada replanejamento, é gerada dinamicamente uma grelha cartesiana local delimitada pelas coordenadas de partida e destino, com uma margem de 3,0 m e resolução configurável (por omissão, 0,5 m por célula). Não existe grelha global persistente: a grelha é construída sob pedido e descartada após a extração do caminho.

5.2 Função de Custo

A expansão de nós minimiza a função clássica:

$$f(n) = g(n) + h(n) \tag{23}$$

onde $g(n)$ é a distância euclidiana acumulada desde a origem (ponderada pela resolução da grelha, incluindo movimentos diagonais) e $h(n)$ é a distância euclidiana em linha reta até ao destino.

5.3 Critério de Exclusão de Obstáculos

Durante a expansão de vizinhos, qualquer nó cuja posição no mundo satisfaça

$$\exists\, o \in \mathcal{O} : \|p_n - o\| < r_s \tag{24}$$

é imediatamente descartado, onde $\mathcal{O}$ é o conjunto de posições bidimensionais de todos os jogadores detetados (adversários e colegas) e $r_s = 0{,}8$ m é o raio de segurança configurado.

5.4 Replanejamento Dinâmico

O caminho é recalculado sempre que o destino global se desloca mais de 0,3 m relativamente ao planeamento anterior, ou quando um obstáculo entra no raio de segurança do *waypoint* corrente. Se o A* não encontrar solução, o agente dirige-se diretamente ao destino como comportamento de *fallback*.

6 Casos de Uso e Avaliação de Risco

6.1 Casos de Uso Multi-Perfil

O ecossistema do projeto foi estruturado para suportar três perfis de interação distintos: utilizadores competitivos finais (operadores de campo), desenvolvedores futuros (investigadores que herdam os ambientes Gymnasium) e membros da equipa atual (integração de código paralela).

6.2 Avaliação e Matriz de Mitigação de Riscos

Abaixo apresenta-se a tabela estruturada com os riscos de desenvolvimento identificados e as respetivas ações preventivas tomadas na engenharia do sistema.

Table 2: Riscos do projeto e estratégias aplicadas de mitigação

Identificação do Risco	**Estratégia Aplicada de Mitigação**
Escassez de recursos computacionais locais para a convergência de múltiplas redes PPO.	Paralelização de ambientes recorrendo ao *SubprocVecEnv*, simplificação temporária da densidade do vetor de estados e agendamento de janelas de treino noturno partilhado.
Instabilidade ou quebras de simulação física durante impactos de alta velocidade no MuJoCo.	Redução do passo de integração temporal do motor de física e desenho de restrições rígidas (*clipping*) nos limites de binário das juntas no espaço de ações.
Dificuldade de acoplamento entre o planeador geométrico A* e os atuadores contínuos do robô.	Desenvolvimento de uma camada de controlo intermédia que traduz os *waypoints* discretos gerados pelo A* em vetores de direção e velocidade linear para a rede de caminhada.

7 Resultados Esperados

Com o desenvolvimento dos comportamentos motores e da camada de navegação, espera-se que a equipa FC Portugal consiga competir de forma eficaz no novo simulador MuJoCo. Os comportamentos treinados deverão demonstrar estabilidade e robustez em situações de jogo reais, e a integração do A* deverá permitir uma navegação em campo fluida e sem colisões com outros jogadores.

A participação no RoboCup 2026, a realizar-se em Incheon,Coreia do Sul , será o principal momento de validação do trabalho desenvolvido e permitirá identificar os pontos a melhorar para edições futuras.

8 Conclusões e Trabalho Futuro

8.1 Conclusões

Ao longo deste projeto foi desenvolvido e integrado um conjunto de comportamentos para a equipa FC Portugal no novo simulador MuJoCo. Os comportamentos de queda, drible, remate curto, remate longo e defesa do guarda-redes foram treinados com sucesso usando o algoritmo PPO, e a navegação em campo foi implementada de forma independente através do algoritmo A*. A transição para o MuJoCo revelou-se desafiante mas necessária, e os resultados obtidos colocam a equipa em boa posição para o RoboCup 2026.

8.2 Trabalho Futuro

Como trabalho futuro, identificam-se as seguintes direções:

- **Melhoria dos comportamentos existentes:** Continuar a ajustar o treino dos comportamentos mais sensíveis, como o drible e o remate longo, para aumentar a sua consistência em jogo.
- **Coordenação entre agentes:** Desenvolver mecanismos de colaboração entre robôs, como passe e posicionamento em equipa.
- **Navegação mais inteligente:** Melhorar a camada A* para antecipar o movimento dos adversários em vez de apenas reagir à sua posição atual.
- **Validação em competição:** A participação no RoboCup 2026 será o principal teste ao sistema desenvolvido e servirá de base para iterações futuras.